

Programación masivamente paralela en procesadores gráficos - Práctico 4: Sincronización por grupos y bibliotecas de primitivas generales

Nery Ezequiel Capó Brazeiro - Grupo 32
Facultad de Ingeniería
Universidad de la República
Montevideo, Uruguay
nery.ezequiel.capo@fing.edu.uy

Miguel Rodrigues - Grupo 32
Facultad de Ingeniería
Universidad de la República
Montevideo, Uruguay
miguelrodriguescas1333@gmail.com

I. INTRODUCCIÓN

El objetivo de este práctico es poner en práctica conceptos relacionados con el uso de Cooperative Groups y bibliotecas de alto desempeño, en particular Thrust y CUB.

II. EJERCICIO 1

II-A. Parte a

Se implemento la rutina de patrón de cómputo Scan (prefix-sum) mediante un enfoque de árbol binario balanceado para optimizar la eficiencia del cálculo paralelo. Para maximizar el rendimiento, se utiliza la shared memory, permitiendo que los hilos de un mismo bloque cooperen de forma eficiente y reduzcan la latencia de acceso a datos

En la primera etapa, el vector de entrada se particiona en secciones para que cada bloque de hilos procese una porción de manera independiente. El scan se realiza en dos fases principales: Etapa de reducción (Up-sweep): Se recorre desde las hojas hacia la raíz del árbol binario, construyendo sumas parciales en los nodos internos hasta que la raíz contiene la suma de los elementos del bloque. Etapa post-reducción (Down-sweep): Se recorre el árbol en sentido inverso para construir los valores de salida finales.

Para manejar vectores cuyo tamaño excede la capacidad de un único bloque, se aplica una estrategia jerárquica de múltiples etapas. Las sumas totales de cada bloque se almacenan en un vector auxiliar global, sobre este vector se realiza un scan adicional para calcular los offsets globales necesarios para cada bloque.

Al final se lanza un kernel adicional que suma estos desplazamientos a todos los elementos locales de cada bloque, asegurando la correctitud del resultado global.

II-B. Parte b

La segunda solución emplea la biblioteca CUB, específicamente la rutina:

cub :: DeviceScan :: ExclusiveSum

Esta función implementa una versión optimizada del algoritmo de scan para GPU. Una vez reservada la cantidad

de memoria temporal que requiere la etapa inicial de CUB, la función puede ejecutarse repetidamente reutilizando el mismo almacenamiento temporal. Con el objetivo de evitar que los costos de inicialización afectaran las mediciones, se realizó una ejecución previa antes de comenzar el registro de tiempos.

La tercera solución utiliza la biblioteca Thrust mediante la primitiva:

thrust :: exclusive_scan

La biblioteca Thrust permite realizar el cálculo del scan exclusivo directamente sobre vectores almacenados en la memoria de la GPU; Para ello se construye un *thrust :: device_vector* a partir del vector de entrada y se reserva un segundo vector para almacenar el resultado. La operación de scan se ejecuta mediante una única llamada a *thrust :: exclusive_scan*, siendo la biblioteca la que realiza la planificación y ejecución de los kernels necesarios.

Al igual que en el caso de CUB, se efectuó una ejecución de calentamiento previa a las mediciones para evitar sesgos producidos por costos de inicialización.

II-C. Parte 3

Cuadro I: Tiempo promedio y desviación estándar del escaneo para tamaños $N = 1024 \times 2^k$, con $k = 1, \dots, 10$.

k	Manual (ms)		CUB (ms)		Thrust (ms)	
	Avg	StdDev	Avg	StdDev	Avg	StdDev
1	0.037	0.003	0.015	0.001	0.023	0.002
2	0.037	0.003	0.015	0.001	0.024	0.002
3	0.042	0.016	0.020	0.010	0.023	0.002
4	0.038	0.004	0.015	0.001	0.025	0.003
5	0.039	0.003	0.016	0.003	0.024	0.002
6	0.041	0.003	0.016	0.001	0.025	0.003
7	0.076	0.013	0.018	0.004	0.027	0.003
8	0.199	0.013	0.019	0.002	0.141	0.005
9	0.227	0.017	0.020	0.001	0.146	0.006
10	0.276	0.009	0.031	0.001	0.157	0.008

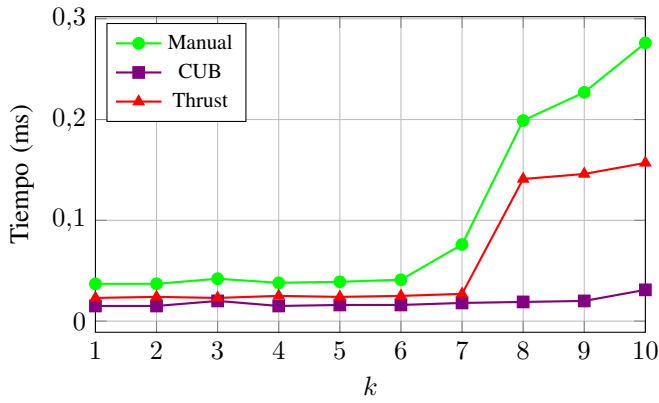


Figura 1: Tiempo promedio del escaneo para $N = 1024 \cdot 2^k$.

Los resultados muestran que la implementación basada en CUB presenta el mejor rendimiento para todos los tamaños evaluados. Este resultado tiene sentido ya que se trata de una biblioteca específicamente diseñada para utilizar las características de las arquitecturas GPU lo más eficiente posible.

La implementación basada en Thrust también obtiene buenos resultados, aunque presenta tiempos superiores a los de CUB para todos los tamaños evaluados. Esta diferencia puede explicarse porque Thrust proporciona una interfaz de más alto nivel y delega internamente la ejecución a primitivas optimizadas, introduciendo cierto overhead adicional respecto al uso directo de CUB. De igual forma su desempeño continúa siendo mejor que el de la implementación manual. La implementación manual presenta los mayores tiempos de ejecución. Aunque utiliza memoria compartida para reducir accesos a memoria global y emplea una estrategia jerárquica eficiente para procesar vectores de longitud arbitraria, requiere múltiples lanzamientos de kernels y etapas adicionales para combinar los resultados parciales de cada bloque. Estos costos adicionales se vuelven más notorios a medida que aumenta el tamaño de la entrada, produciendo un crecimiento más pronunciado en los tiempos de ejecución.

Para el mayor tamaño evaluado siendo $N = 1024 \cdot 2^{10}$, la implementación manual alcanza en promedio 0.276 ms, mientras que Thrust requiere en promedio 0.157 ms y CUB solamente en promedio 0.031 ms. Esto evidencia el elevado grado de optimización presente en las bibliotecas especializadas, particularmente en CUB.

Para tamaños pequeños las diferencias entre implementaciones son reducidas debido a que predominan los costos de lanzamiento de kernels. Pero al aumentar el tamaño de entrada la diferencia entre la implementación manual y las bibliotecas optimizadas se vuelve mucho más significativa.

III. EJERCICIO 2: USO DE COOPERATIVE GROUPS

El ejercicio se compone de dos partes, y esta basado en [2] y contenido teórico visto en clase.

III-A. Parte 1

En la primera parte se emplean *Cooperative Groups* para realizar reducciones sobre segmentos contiguos de ocho elementos. Para ello se utiliza `cooperative_groups::tiled_partition< 8 >`, particionando cada bloque en grupos de ocho hilos. Posteriormente se aplica la operación `reduce(coop_g, valor_hilo_sumar, cg::plus<int>())` provista por la biblioteca sobre cada grupo, obteniendo la suma de los ocho elementos correspondientes y almacenando el resultado en el vector de salida.

III-B. Parte 2

En la segunda parte se utiliza la funcionalidad de *Labeled Partition*. Para ello se construye un vector auxiliar de índices, donde cada índice se repite durante N posiciones consecutivas (siendo $N = 4$ el valor utilizado por defecto) antes de incrementar su valor. De esta manera se generan segmentos contiguos de elementos que comparten la misma etiqueta.

A continuación, dichos segmentos son identificados mediante `labeled_partition`, que agrupa los hilos que poseen una misma etiqueta. La operación se realiza sobre grupos de 32 hilos. Esta decisión se encuentra alineada con las restricciones establecidas. De tal manera, se intuye que `labeled_partition`, genera los grupos sin necesidad de coordinar hilos pertenecientes a distintos warps.

IV. EJERCICIO 3: USO DE *Building Blocks* PARA PARALELIZAR RUTINAS COMPLEJAS

Para la resolución de este ejercicio se emplearon diversos *building blocks* de la librería Thrust con el objetivo de implementar la lógica requerida aprovechando el paralelismo masivo disponible en la GPU. Una de las principales ventajas de Thrust es que dispone una interfaz de alto nivel, que permite abstraer la implementación y gestión de memoria.¹

La implementación se desarrolló utilizando primitivas vistas en las presentaciones del curso y complementando la información con la documentación oficial de NVIDIA. En una primera etapa se empleó `transform` [6] para calcular el bin asociado a cada elemento del arreglo de entrada mediante una operación unaria personalizada.

Posteriormente se utilizó `stable_sort_by_key` [5], que permite ordenar los elementos según el bin al que pertenecen. La variante *stable* preserva el orden relativo de elementos con la misma clave. Esto implica que, si dos

¹Sin embargo es más eficiente liberar la memoria de manera manual, visto en teórico

Cuadro II: Implementación paralela utilizando Thrust.

N	Avg (ms)	StdDev (ms)
10^3	0.158	0.003
10^4	0.257	0.015
10^5	0.363	0.008
10^6	1.505	0.013
10^7	12.460	0.024

Cuadro III: Implementación secuencial (CodigoBins.cu).

N	Avg (ms)	StdDev (ms)
10^3	0.003	0.001
10^4	0.071	0.012
10^5	0.837	0.080
10^6	10.402	0.500
10^7	128.137	1.545

elementos pertenecen al mismo bin y uno aparece antes que el otro en la entrada original, dicha relación se mantiene luego del ordenamiento.

Una vez ordenados los datos, la cantidad total de bins se determina según el bin de mayor valor más uno (ya que los bins estan ordenados). Para ello se obtiene el último elemento mediante `.back()` y se incrementa su valor en una unidad, dado que los bins se indexan desde cero.

El cálculo de `bin_count` se realizó mediante `reduce_by_key` [4], utilizando un iterador constante de valor uno. De esta forma, la reducción acumula la cantidad de ocurrencias asociadas a cada bin.²

Finalmente, los desplazamientos (`bin_offsets`) se obtuvieron aplicando `exclusive_scan` sobre los `bin_count`. Una vez completada la implementación, se compararon los tiempos de ejecución de la versión paralela basada en Thrust con una implementación secuencial de referencia para distintos tamaños de entrada.

Los resultados obtenidos coinciden con lo esperados. Para tamaños de entrada pequeños, la versión secuencial presenta mejores tiempos debido a los costos asociados a la ejecución y definición de uso de la GPU, como: Transferencias de memoria, sincronizaciones, etc.

Sin embargo, a medida que aumenta el tamaño de N . A partir de $N \approx 10^5$ puede observarse que la versión paralela comienza a superar a la implementación secuencial, alcanzando para $N = 10^7$ una aceleración superior a un orden de magnitud. Este comportamiento refleja adecuadamente la naturaleza de las arquitecturas GPU, diseñadas para explotar paralelismo de datos a gran escala.

La implementación secuencial presenta un crecimiento aproximadamente lineal respecto del tamaño de entrada. Esto

²La estrategia es equivalente a utilizar un vector compuesto únicamente por unos, como se muestra en ejemplos de la documentación oficial, pero evita almacenamiento adicional y resulta más eficiente, también se descarta las claves

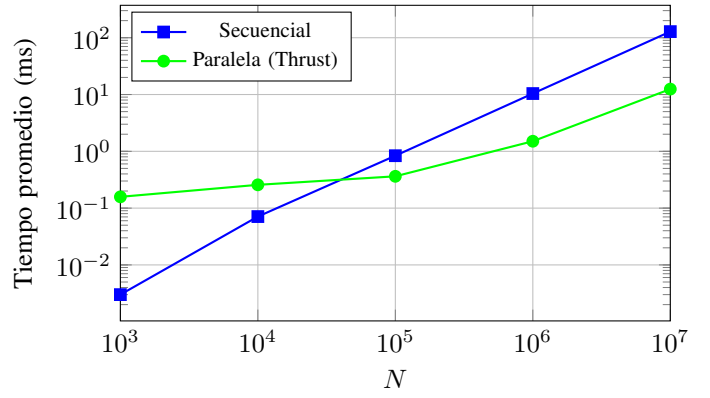


Figura 2: Comparación t. promedio de ejecución secuencial y paralela(Thrust). (Escala log)

resulta consistente tanto con el análisis del código como con los resultados experimentales, por lo que puede modelarse razonablemente mediante $O(N)$.

En contraste, determinar la complejidad de la versión paralela no resulta inmediato. La implementación se encuentra compuesta por múltiples primitivas de Thrust. Y el empleo de paralelismo dificulta más su análisis. Pero puede deducirse que no es lineal, por los datos experimentales.

V. CONCLUSIONES

Se cumplieron los objetivos planteados en el práctico. Se implementó y analizó una versión manual del patrón scan comparándola con las soluciones provistas por las bibliotecas CUB y Thrust, lo que permitió evidenciar las ventajas de utilizar primitivas altamente optimizadas.

A su vez, se exploró el uso de Cooperative Groups* para la sincronización y organización de hilos, empleando mecanismos de particionamiento y reducción provistos por bibliotecas de alto nivel.

Finalmente, se utilizaron building blocks de Thrust para desarrollar una solución paralela de mayor complejidad, permitiendo abordar un problema complejo mediante primitivas de alto nivel y obteniendo tiempos de ejecución competitivos. Los resultados experimentales evidenciaron el impacto positivo del paralelismo masivo sobre el rendimiento para tamaños de entrada suficientemente grandes.

En conjunto, los resultados obtenidos permitieron validar los conceptos teóricos vistos en el curso y comprender las ventajas que ofrecen las abstracciones modernas para el desarrollo eficiente de aplicaciones GPGPU. Estas herramientas simplifican significativamente el desarrollo sin renunciar al rendimiento, constituyendo una alternativa de gran valor en la programación sobre GPU.

En particular, CUB ofrece un mayor nivel de control y el mejor desempeño entre las alternativas evaluadas, a costa

de una interfaz de menor nivel. Por su parte, Thrust permite abordar problemas complejos mediante abstracciones de muy alto nivel, simplificando considerablemente la implementación aunque con una ligera pérdida de rendimiento. Aun así, las implementaciones manuales continúan siendo relevantes, ya que permiten comprender en profundidad el funcionamiento de los algoritmos y adaptar las soluciones a necesidades específicas, aunque implican un mayor esfuerzo de desarrollo y mantenimiento.

VI. ANEXOS

Se modificó CódigoBins.cu para calcular 10 ejecuciones. Las ejecuciones con librerías se descarto la primera ejecución (warm - up) Se calculan los tiempos con `nvtxRangePush`, `nvtxRangePop` Y `nsys`.

Los casos de prueba se ejecutan con `bash prueba.sh`, el tiempo de ejecución puede ser considerado elevado 6 minutos aprox. Se prueba funcionalidad y tiempos de ejecución (menos para ejercicio 2, solo funcionalidad).

A razón de facilitar la ejecución, Se brindan salidas ya ejecutadas, y se muestran con `bash salidas.sh` (Lo cual es opcional de ver, simplemente para no esperar la ejecución de todos los casos de prueba).

REFERENCIAS

- [1] P. Ezzatti, M. Pedemonte, E. Dufrechou, M. Freire, Facultad de Ingeniería - Universidad de la República, https://eva.fing.edu.uy/pluginfile.php/98384/mod_label/intro/Clase%2012%20-%20Abstracciones%20y%20Bibliotecas%20en%20CUDA%20Moderno.pdf [Online].
- [2] NVIDIA Corporation, *CUDA Programming Guide: 4.4 Cooperative Groups*, Disponible en: <https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/cooperative-groups.html>, [Online].
- [3] NVIDIA Corporation, *CUDA Programming Guide: 5.6 Device-Callable APIs and Intrinsics*, Disponible en: <https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/device-callable-apis.html>, [Online].
- [4] NVIDIA Corporation, *CUDA Core Compute Libraries - thrust::reduce_by_key*, Disponible en: https://nvidia.github.io/cccl/unstable/thrust/api/group__reductions_1gab66d365f9a531bb5b99615ac3e836f8f.html.
- [5] NVIDIA Corporation, *CUDA Core Compute Libraries - thrust::stable_sort_by_key*, Disponible en: https://nvidia.github.io/cccl/unstable/thrust/api/group__sorting_1ga289fae2c9edaceba94c025beeb615ca2.html.
- [6] NVIDIA Corporation, *CUDA Core Compute Libraries - thrust::transform*, Disponible en: https://nvidia.github.io/cccl/unstable/thrust/api/group__transformations_1gabbd6380c902223d777cc72d3b1b9d1a.html.